



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
Проектування та дослідження програмних систем
зі штучним інтелектом

Тема: «Реалізація та дослідження рекурентної нейронної мережі LSTM для роботи з текстовими даними»

Виконала
студентка групи ІК-51мп:
Яковенко Д. О.

ЗМІСТ

Практичне завдання	2
Хід роботи	3
Завантаження та первинний огляд даних	3
Попередня обробка тексту.....	4
Дослідження методів токенізації	5
Побудова, навчання та оцінка нейромереж LSTM.....	6
Аналіз результатів та матриць помилок.....	7
Тестування моделі на реальних текстах	10
Посилання на код:	11
Відповіді на питання	11
Висновки	13

Практичне завдання

Завдання: Написати програму, що реалізує рекурентну нейронну мережу LSTM для розпізнавання аналізу настроїв тексту, використати датасет yelp review polarity dataset.

Завдання на лабораторну роботу:

1. Завантажити набір даних yelp review polarity dataset.
2. Провести попередню обробку текстів для навчання нейронної мережі.
3. Запрограмувати архітектуру нейронної мережі LSTM.
4. Навчити нейронну мережу LSTM.
5. Оцінити правильність роботи мережі, побудувати матрицю помилок, обрахувати значення accuracy, precision, recall та F-Score.
6. Провести тестування моделі – написати програму аналізу настроїв навченою нейронною мережею. Провести тестування на текстах з навчального набору даних так і з аналогічними за параметрами текстами, що не входять до набору даних.
7. Додаткове завдання на +1 бал, проаналізувати та реалізувати існуючі на сьогодні методи токенізації текстів для навчання нейронних мереж. Провести дослідження та порівняти точність роботи нейронної мережі за пунктом 5 завдання, з використанням різних методів токенізації(з різних бібліотек).

Хід роботи

Завантаження та первинний огляд даних

У першому блоці коду було завантажено набір даних **Yelp Review Polarity**.

```
import pandas as pd
import numpy as np

# Шляхи до файлів
train_path = '/kaggle/input/datasets/irustandi/yelp-review-polarity/yelp_review_polarity_csv/train.csv'
test_path = '/kaggle/input/datasets/irustandi/yelp-review-polarity/yelp_review_polarity_csv/test.csv'

# Завантажуємо тестові та тренувальні дані
train_df = pd.read_csv(train_path, names=['class', 'text'])
test_df = pd.read_csv(test_path, names=['class', 'text'])

# Розмір датасету
print(f"Розмір тренувальної вибірки: {train_df.shape}")
print(f"Розмір тестової вибірки: {test_df.shape}\n")
display(train_df.head())
```

Рис. 1. – Завантаження набору даних

Використано бібліотеку pandas для зчитування .csv файлів. Оскільки датасет постачається без заголовків, було примусово задано назви колонок: class (оцінка) та text (сам відгук). Встановлено розмірності датасету: 560 000 записів для тренувальної вибірки та 38 000 для тестової. Оригінальні мітки класів були представлені цифрами 1 (негативний) та 2 (позитивний).

Розмір тренувальної вибірки: (560000, 2)
Розмір тестової вибірки: (38000, 2)

	class	text
0	1	Unfortunately, the frustration of being Dr. Go...
1	2	Been going to Dr. Goldberg for over 10 years. ...
2	1	I don't know what Dr. Goldberg was like before...
3	1	I'm writing this review to give you a heads up...
4	2	All the food is great here. But the best thing...

Рис. 2. – Первинний вигляд та розмір датасету

Попередня обробка тексту

Для ефективного навчання нейронної мережі "сирий" текст необхідно очистити від інформаційного шуму.

- Мітки класів перетворено у бінарний формат [0, 1] за допомогою методу `.map()`, що є необхідною умовою для використання функції активації Sigmoid та функції втрат `binary_crossentropy`.
- Написано функцію `clean_text`, яка використовує регулярні вирази для переведення тексту в нижній регістр, видалення HTML-тегів, чисел, знаків пунктуації та системних символів перенесення рядка (`\n`, `\r`).
- Створено зменшені вибірки з метою використання меншої кількості RAM.

```
import re

# Заміна міток класів
class_mapping = {1: 0, 2: 1}
train_df['class'] = train_df['class'].map(class_mapping)
test_df['class'] = test_df['class'].map(class_mapping)

print("Унікальні класи тепер:", train_df['class'].unique())

# Функція для очищення тексту
def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)
    text = text.replace('\n', ' ').replace('\r', ' ')
    text = re.sub(r'^a-z\s', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

SAMPLE_SIZE = 150000
TEST_SIZE = 30000
train_sampled = train_df.sample(n=SAMPLE_SIZE, random_state=42).copy()
test_sampled = test_df.sample(n=TEST_SIZE, random_state=42).copy()
train_sampled['clean_text'] = train_sampled['text'].apply(clean_text)
test_sampled['clean_text'] = test_sampled['text'].apply(clean_text)
```

Рис. 3. – Код обробки тексту

Унікальні класи тепер: [0 1]

Текст очищено

=====
КЛАС: 1
=====

ОРИГІНАЛЬНИЙ ТЕКСТ:

This place is one of my favorite comic shops. I actually live closer to a different one, but I drive to chandler just to go to this one. I like their selection and when they have the dollar sale you can get some ridiculous deals. The staff is ridiculously friendly and I usually always walk out with whatever I wanted. They also have some cool events from time to time and i've found their prices to be reasonable and comparable to other comic shops.

ОЧИЩЕНИЙ ТЕКСТ:

this place is one of my favorite comic shops i actually live closer to a different one but i drive to chandler just to go to this one i like their selection and when they have the dollar sale you can get some ridiculous deals the staff is ridiculously friendly and i usually always walk out with whatever i wanted they also have some cool events from time to time and ive found their prices to be reasonable and comparable to other comic shops

Рис. 4. – Результат обробки

Дослідження методів токенизації

Для виконання розширеного дослідження було реалізовано підготовку даних за допомогою трьох різних методів векторизації тексту.

- **Word-level (По словах):** Використано TextVectorization від Keras. Словник обмежено до 10 000 найпопулярніших слів. Кожне слово представлено унікальним індексом.
- **Character-level (По символах):** Використано параметр `split='character'`. Словник складає всього ~50 символів (алфавіт + пробіл). Цей метод генерує дуже довгі послідовності (до 800 токенів на відгук).
- **Subword-level (По підсловах):** Залучено бібліотеку Hugging Face та токенизатор моделі `bert-base-uncased`. Метод ефективно обробляє невідомі слова, розбиваючи їх на логічні частини, комбінуючи переваги двох попередніх підходів.

Усі підготовлені набори даних збережено у єдиний словник `experiment_data` для автоматизації експерименту.

```
1. Підготовка Word-level токенизації (Keras)...
I0000 00:00:1776932544.434400      55 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13
757 MB memory: -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1776932544.440566      55 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 13
757 MB memory: -> device: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
2. Підготовка Character-level токенизації (Keras)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
3. Підготовка Subword-level токенизації (Hugging Face)...
config.json: 100% ██████████ 570/570 [00:00<00:00, 63.4kB/s]
tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 4.78kB/s]
vocab.txt: █████ 232k/? [00:00<00:00, 9.28MB/s]
tokenizer.json: █████ 466k/? [00:00<00:00, 3.77MB/s]

Всі три методи успішно підготовлені та упаковані в словник experiment_data.

=====
🔍 ПРИКЛАДИ ТОКЕНІЗАЦІЇ
Оригінальний очищений текст: this place is one of my favorite comic shops i actually live closer to a different one but i dri
ve to chandler just to g ...
=====

Метод [Word]:
[ 16  31  12  44   8  14 296 6696 1141   4 233  503 1517   5
  6 236  44  18   4 520]

Метод [Char]:
[ 4 11  7  9  2 21 13  5 15  3  2  7  9  2  6  8  3  2  6 19]

Метод [Subword]:
[ 101 2023 2173 2003 2028 1997 2026 5440 5021 7340 1045 2941 2444 3553
 2000 1037 2367 2028 2021 1045]
```

Рис. 4. – Приклад токенизації тексту кожним з методів

Побудова, навчання та оцінка нейромереж LSTM

Створено універсальну функцію *build_lstm_model*, яка динамічно генерує архітектуру під розміри конкретного токенизатора.

- **Архітектура:**

- Input – шар для прийому послідовностей фіксованої довжини.
- Embedding – перетворює індекси токенів на щільні вектори розмірністю 64.
- LSTM (64 блоки) – рекурентний шар для виявлення довгострокових залежностей у тексті.
- Dropout (0.5) – шар регуляризації для запобігання перенавчанню шляхом випадкового вимкнення 50% нейронів.
- Dense (1 нейрон, Sigmoid) – вихідний шар для бінарної класифікації.

```
# Функція створення мережі під відповідний метод токенизації
def build_lstm_model(vocab_size, max_length):
    model = Sequential([
        Input(shape=(max_length,)),
        Embedding(input_dim=vocab_size, output_dim=64),
        LSTM(64, return_sequences=False),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

early_stop = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
results = []

print("Починаємо експеримент...\n")

# Цикл по трьох методах
for method_name, data in experiment_data.items():
    print(f"{'='*80}")
    print(f"МЕТОД: {method_name}")
    print(f"{'='*80}")

    # Створюємо модель
    model = build_lstm_model(data['vocab_size'], data['max_len'])

    start_time = time.time()

    history = model.fit(
        data['X_train'], y_train,
        epochs=15,
        batch_size=128,
        validation_split=0.2,
        callbacks=[early_stop],
        verbose=1
    )

    train_time = time.time() - start_time
    experiment_data[method_name]['model'] = model
```

Рис. 12. – Код функції генерації архітектури LSTM під метод токенизації

Аналіз результатів та матриць помилок

Згідно з отриманими в ході виконання скрипту графіками та підсумковою таблицею, результати дослідження наступні:

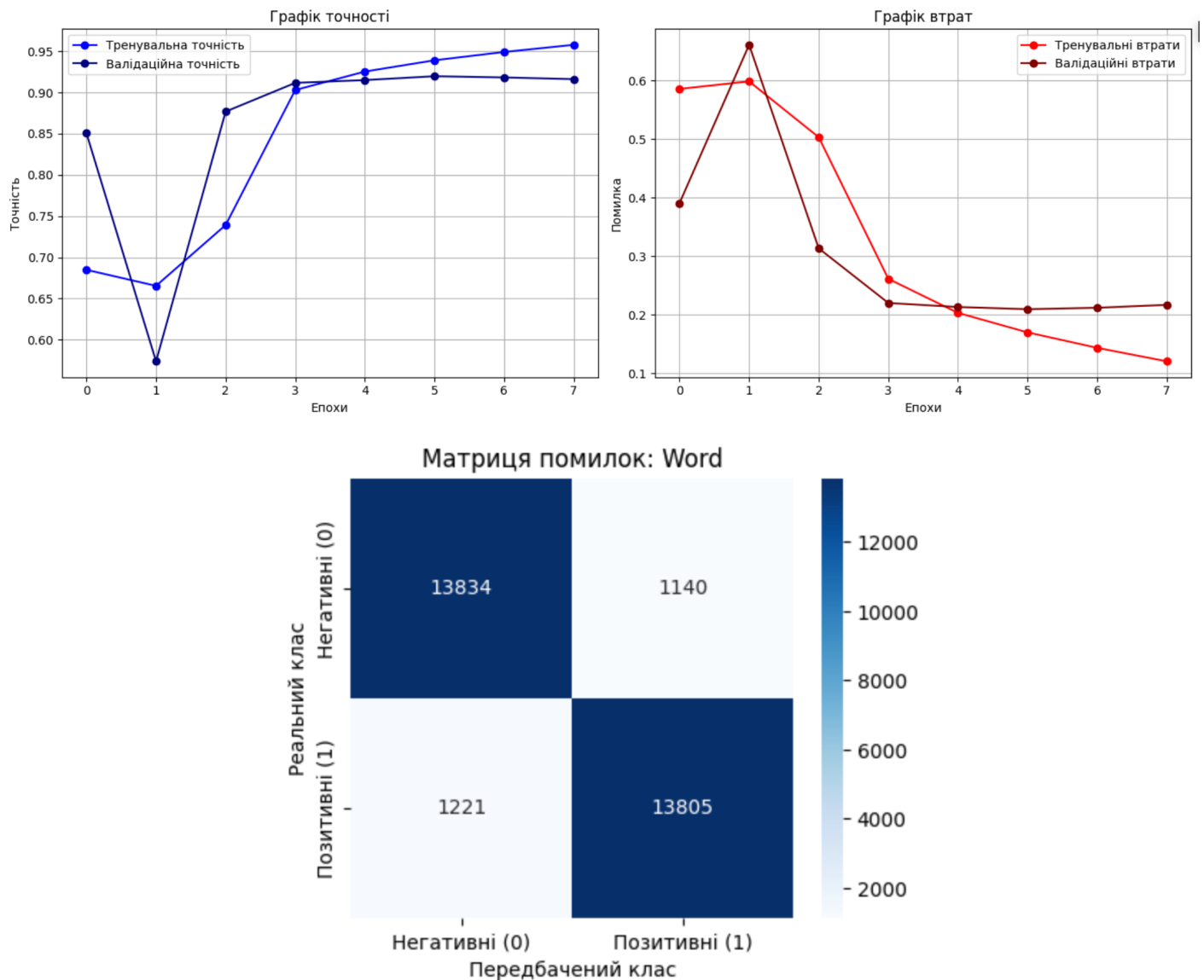


Рис. 12. – По словах токенизація

- **Модель 1:** Показала відмінний результат з **Accuracy** 0.9213 та **F1-Score** 0.9212. Графіки навчання демонструють стабільне падіння втрат. Матриця помилок симетрична: 13834 правильно розпізнаних негативних і 13805 позитивних відгуків. Хибних спрацювань мінімум.

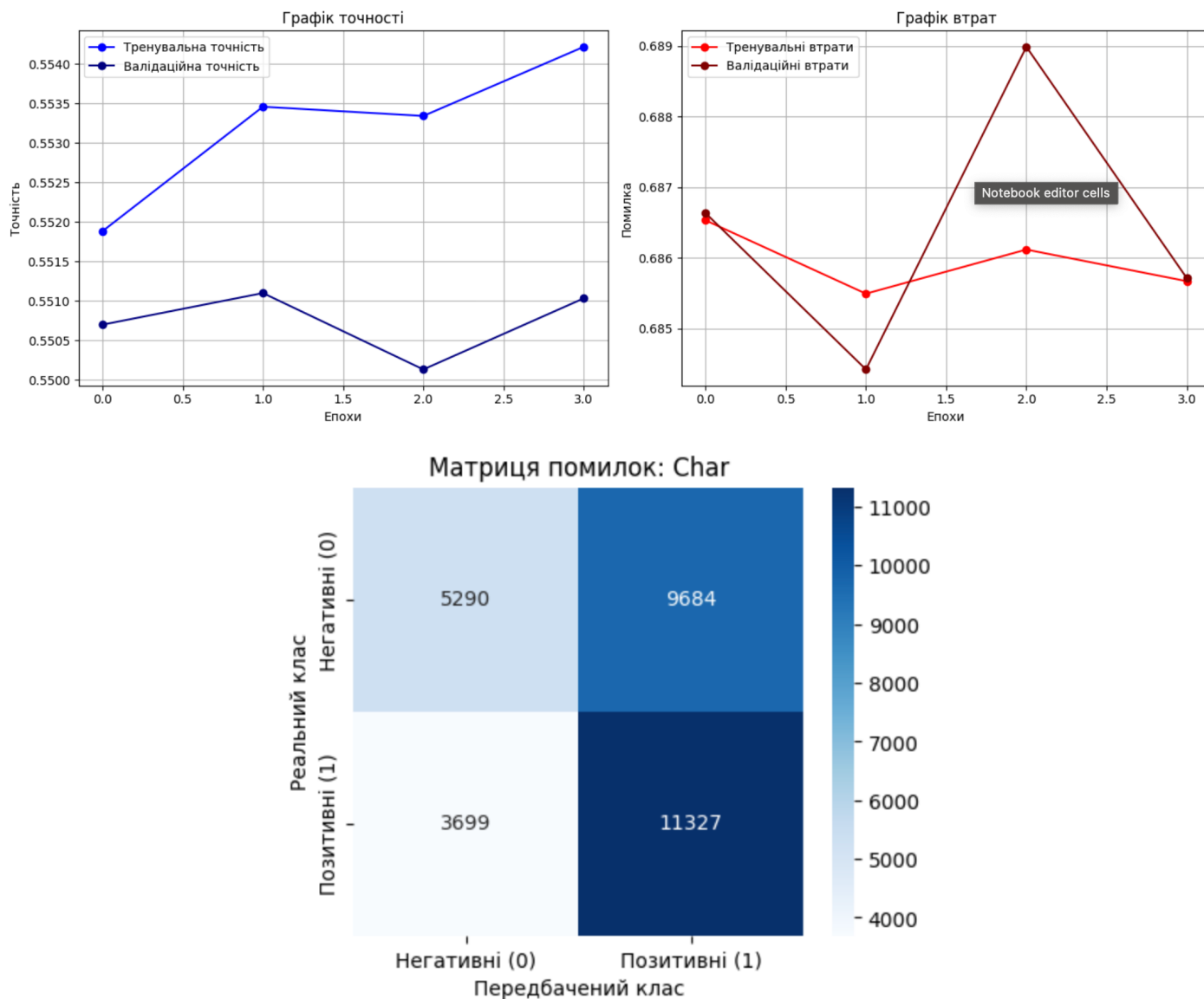


Рис. 12. – По символах токенизація

- **Модель 2:** Показала найгірший результат з **Accuracy 0.5539**. Як видно з матриці помилок, модель сильно схиляється до прогнозування класу 1 (позитивний), даючи величезну кількість False Positives.

Висновок: архітектурі LSTM вкрай складно тримати контекст на наддовгих послідовностях з окремих літер.

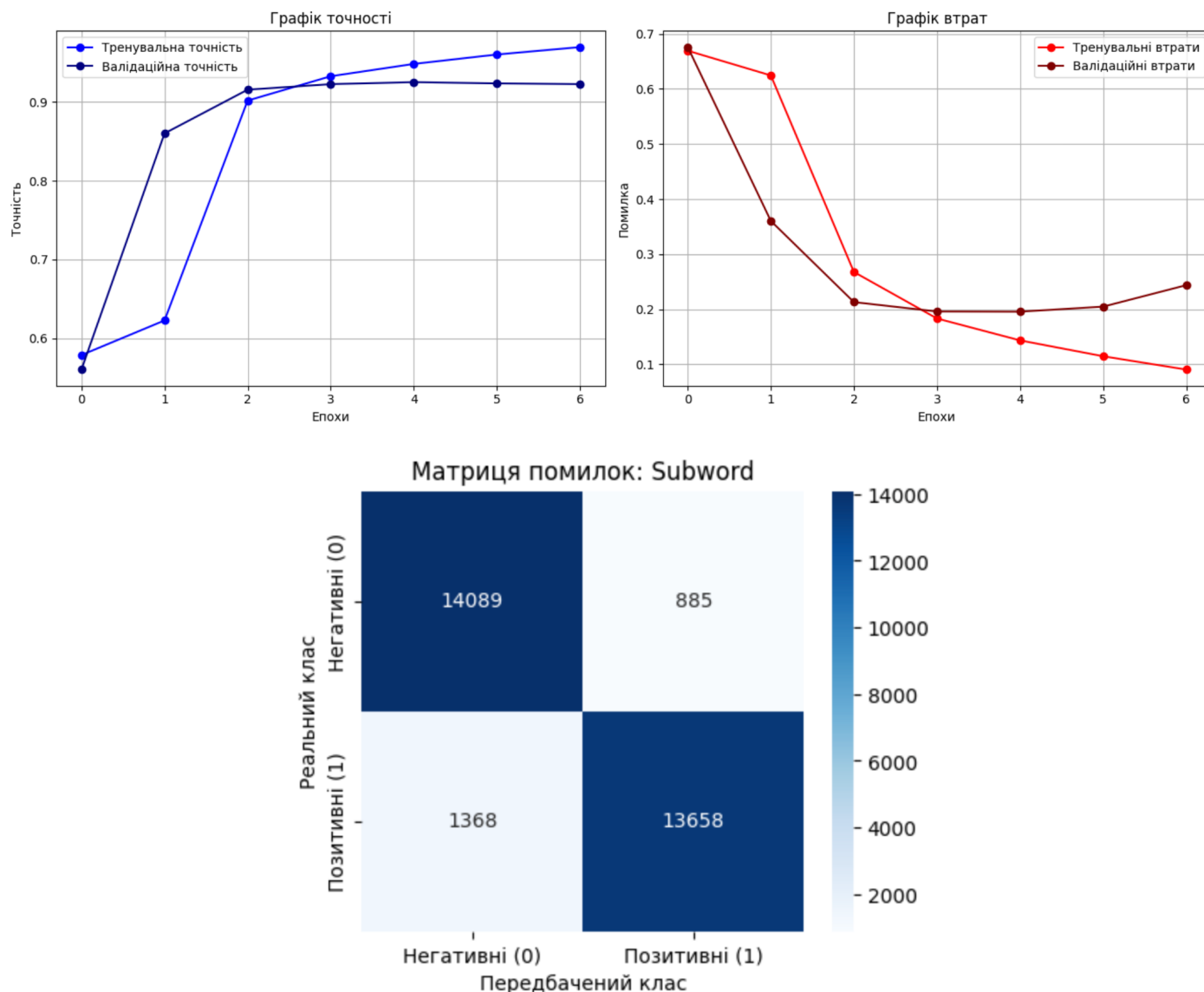


Рис. 12. – По частинах слів токенизація

- **Модель 3:** Продемонструвала найкращу точність на тестових даних – **Accuracy 0.9249** та **Precision 0.9391**. Це підтверджує, що сучасні підходи розбиття тексту на підслова (BERT tokenizer) працюють стабільніше та точніше за базове по-слівне розбиття.

ПІДСУМКОВІ РЕЗУЛЬТАТИ ДОСЛІДЖЕННЯ

	Метод	Accuracy	Precision	Recall	F1-Score	Час (сек)
0	Word	0.9213	0.9237	0.9187	0.9212	83.8
1	Char	0.5539	0.5391	0.7538	0.6286	136.0
2	Subword	0.9249	0.9391	0.9090	0.9238	88.6

Рис. 12. – По частинах слів токенизація

Тестування моделі на реальних текстах

Для перевірки роботи навчених нейромереж у "бойових" умовах було написано функцію *analyze_my_text*. Було протестовано 2 записи з оригінального датасету та 3 власні відгуки, яких не було в навчальній вибірці.

Мережі (особливо Word та Subword) успішно впоралися з класифікацією очевидних настроїв.

На змішаних відгуках (де присутні слова і з позитивним, і з негативним забарвленням) можна було спостерігати різницю у впевненості між різними підходами до токенизації.

ТЕСТУВАННЯ НА ТЕКСТАХ З ДАТАСЕТУ

```
=====
📄 ТЕКСТ: 'After a bad experience at our Favorite Restaurant Roca Akor in Scottsdale, we turned to Yelp and Roca's very capable manager Charles responded within an hour, invited us back and more than made it right. We knew they had great food and now, thanks to Charles, we know they have great management. We...'
```

Word-level LSTM:	0.7531	->	🟢	ПОЗИТИВНИЙ
Char-level LSTM:	0.5368	->	🟢	ПОЗИТИВНИЙ
Subword-level LSTM:	0.9620	->	🟢	ПОЗИТИВНИЙ

```
=====
📄 ТЕКСТ: 'This is the the most dishonest car shop I have ever been to, utterly disgusted at the conduct and lack of ethics of the employees who work here.\n\nA few weeks ago I noticed that two of the tail lights on my Acura were out. I took it in Midas as they were the closest shop and knew they were open on ...'
```

Word-level LSTM:	0.0023	->	🔴	НЕГАТИВНИЙ
Char-level LSTM:	0.5368	->	🟢	ПОЗИТИВНИЙ
Subword-level LSTM:	0.0442	->	🔴	НЕГАТИВНИЙ

Рис. 12. – Класифікація настрою на реченнях з датасету

ТЕСТУВАННЯ НА ВЛАСНИХ ТЕКСТАХ

```
=====
📄 ТЕКСТ: 'I recently moved near Bang Sare and this local seafood restaurant is absolutely amazing! The staff is so friendly and the fish is incredibly fresh. Highly recommend it.'
```

Word-level LSTM:	0.9983	->	🟢	ПОЗИТИВНИЙ
Char-level LSTM:	0.5368	->	🟢	ПОЗИТИВНИЙ
Subword-level LSTM:	0.9978	->	🟢	ПОЗИТИВНИЙ

```
=====
📄 ТЕКСТ: 'I bought a new laser tape measure for my interior design project, but it is completely useless and inaccurate. The battery died after one hour. Terrible experience.'
```

Word-level LSTM:	0.0023	->	🔴	НЕГАТИВНИЙ
Char-level LSTM:	0.5368	->	🟢	ПОЗИТИВНИЙ
Subword-level LSTM:	0.0068	->	🔴	НЕГАТИВНИЙ

```
=====
📄 ТЕКСТ: 'The flight was extremely long and exhausting, but the service on the airplane was exceptional and the food was surprisingly good.'
```

Word-level LSTM:	0.9620	->	🟢	ПОЗИТИВНИЙ
Char-level LSTM:	0.5368	->	🟢	ПОЗИТИВНИЙ
Subword-level LSTM:	0.9864	->	🟢	ПОЗИТИВНИЙ

Рис. 12. – Класифікація настрою на власних реченнях

Посилання на код:

<https://www.kaggle.com/code/darrriax/lab6ai>

Відповіді на питання

1. Що таке обробка природної мови (NLP) і яку роль у ній відіграють нейронні мережі?

NLP (Natural Language Processing) — це напрямок штучного інтелекту, який навчає комп'ютери розуміти людську мову. Нейромережі тут відіграють роль аналітичного центру: вони не просто шукають слова за словником, а здатні розуміти контекст, емоційне забарвлення та семантичні зв'язки між словами завдяки їх перетворенню у математичні вектори.

2. У чому полягає головна відмінність між звичайними штучними нейронними мережами та рекурентними нейронними мережами (RNN)?

Головна різниця — у наявності "пам'яті". Звичайні мережі (наприклад, Feed-Forward) обробляють кожен вхідний сигнал ізольовано. RNN мають зворотні зв'язки (loops), завдяки яким інформація з попереднього кроку зберігається у прихованому стані і впливає на обробку поточного елемента послідовності.

3. Чому стандартні RNN погано працюють з довгими текстовими послідовностями?

Вони страждають від проблеми "короткострокової пам'яті". Поки стандартна RNN доходить до кінця довгого речення чи абзацу, вона майже повністю втрачає (забуває) інформацію про перші слова через математичну проблему затухання градієнта.

4. Що таке проблема зникання та вибухання градієнтів у RNN?

Це проблема, що виникає під час навчання мережі (метод зворотного поширення помилки). Оскільки RNN множить градієнти (помилки) на одні й ті самі ваги багато разів, то якщо ваги менші за одиницю, градієнт стрімко зменшується до нуля (зникає), і перші шари не навчаються. Якщо ваги більші за одиницю, градієнт росте до безмежності (вибухає), і мережа видає помилку.

5. Як архітектура LSTM вирішує проблему зникання градієнтів?

LSTM має так званий "стан комірки" (cell state), який проходить через увесь блок як прямий конвеєр. Інформація на цьому конвеєрі оновлюється здебільшого через операцію додавання, а не множення. Це дозволяє градієнтам безперешкодно протікати від кінця до самого початку тексту без затухання.

6. Які основні компоненти має блок LSTM?

LSTM складається зі стану комірки (довгострокової пам'яті) та трьох вентилів (gates):

Вентиль забування (Forget Gate): вирішує, яку стару нерелевантну інформацію стерти.

Вхідний вентиль (Input Gate): вирішує, яку нову інформацію з поточного кроку зберегти.

Вихідний вентиль (Output Gate): визначає, яку частину пам'яті передати на вихід як результат для наступного кроку.

7. У чому полягає відмінність між LSTM та GRU?

GRU (Gated Recurrent Unit) — це спрощена, більш сучасна версія LSTM. У GRU немає окремого стану комірки (усе зберігається в прихованому стані), а вентиля входу та забування об'єднані в один (Update Gate). Завдяки цьому GRU навчається швидше і споживає менше ресурсів, хоча на дуже складних задачах LSTM іноді працює трохи точніше.

8. Як саме LSTM дозволяють моделі запам'ятовувати важливу інформацію протягом довгого часу?

Завдяки вентилям мережа сама вчиться, що важливо, а що ні. Якщо LSTM бачить новий підмет у реченні, вентиль забування стирає старий підмет, а вхідний вентиль записує новий у стан комірки. Цей запис надійно зберігається на конвеєрі доти, доки не стане потрібним для формування контексту.

9. У яких завданнях обробки тексту найчастіше використовують LSTM?

Найчастіше їх застосовують для аналізу настроїв (sentiment analysis), розпізнавання іменованих сутностей (NER), машинного перекладу, створення чат-ботів, автодоповнення тексту та генерації відповідей.

10. Яким чином RNN та LSTM застосовуються для машинного перекладу?

Використовується архітектура Encoder-Decoder (кодувальник-декодувальник). Перша LSTM (Encoder) послідовно "читає" текст оригіналу і стискає його зміст у єдиний вектор контексту. Друга LSTM (Decoder) отримує цей вектор і крок за кроком генерує переклад іншою мовою.

11. Що таке двонаправлені RNN (BiRNN) і чому вони покращують якість обробки тексту?

BiRNN обробляють текст одночасно у двох напрямках: одна мережа читає зліва направо, а інша — справа наліво (з кінця). Це значно покращує якість, бо для розуміння поточного слова мережа "бачить" не лише те, що було сказано раніше, а й те, що буде сказано далі.

12. Як поєднання word embeddings (Word2Vec або GloVe) з LSTM покращує розуміння текстів?

Embeddings замінюють сухі індекси слів на щільні математичні вектори, де слова зі схожим значенням мають схожі координати. LSTM отримує на вхід не просто цифри, а вже готовий "семантичний сенс" слів, що дозволяє їй швидше знаходити логічні закономірності в тексті.

13. Чому LSTM підходять для завдань аналізу настроїв у тексті?

Настрій часто залежить від контексту та заперечень, які стоять далеко один від одного. Наприклад, у фразі "The food was not completely terrible", LSTM пам'ятає частку "not" з початку і правильно інвертує значення слова "terrible" в кінці, визначаючи відгук як швидше позитивний/нейтральний.

14. Як можна використати рекурентні нейронні мережі для розпізнавання іменованих сутностей (NER)?

Для NER зазвичай використовують BiLSTM. Мережа аналізує кожне слово, враховуючи попередній і наступний контекст, і для кожного кроку (слова) видає класифікацію: чи це звичайне слово, ім'я людини (Person), локація (Location), чи назва організації (Organization).

15. Чому для обробки дуже довгих текстів нині частіше використовуються трансформери, а не LSTM?

Трансформери (як BERT чи GPT) використовують механізм Уваги (Self-Attention) і обробляють усі слова паралельно, а не крок за кроком, як LSTM. Це робить їх в рази швидшими при навчанні на GPU і дозволяє утримувати контекст цілих книг, тоді як LSTM все одно починає "забувати" інформацію після кількох сотень слів.

16. Які переваги має LSTM перед звичайними RNN у задачах генерації текстів?

При генерації тексту критично важливо утримувати логіку. Якщо на початку речення був підмет у множині, дієслово через 15 слів також має бути у множині. Звичайна RNN забуде про множину і зробить граматичну помилку, а LSTM збереже цю деталь у стані комірки і згенерує правильне слово.

17. Як відбувається навчання LSTM на великих текстових корпусах?

Навчання йде алгоритмом BPTT (Backpropagation Through Time). Текст розбивається на пакети (батчі). Мережа намагається передбачити наступні слова, порівнює свій прогноз із реально правильним словом, розраховує помилку (Loss) і оновлює свої ваги. Через послідовну природу LSTM цей процес вимагає багато часу та потужних відеокарт.

18. Чи можливо комбінувати LSTM із згортковими мережами (CNN) для обробки текстів?

Так, ця архітектура називається C-LSTM. Вона дуже ефективна: спочатку 1D-CNN працює як фільтр і швидко знаходить важливі локальні патерни (наприклад, ключові словосполучення), а потім LSTM отримує ці "стислі" ознаки і аналізує їхню послідовність у часі. Це пришвидшує роботу.

19. Які обмеження та недоліки мають LSTM у сучасних NLP-системах?

Головний недолік — неможливість розпаралелювання. Оскільки LSTM обробляє текст суворо послідовно (щоб обробити 10-те слово, треба дочекатися результату 9-го), вони тренуються дуже повільно. Також, незважаючи на розв'язання проблеми градієнтів, їхня контекстна пам'ять все ж обмежена порівняно з Трансформерами.

20. Як можна застосувати рекурентні нейронні мережі для передбачення наступного слова в реченні?

Мережі подається послідовність з N слів (наприклад, "Я йду в..."). LSTM пропускає ці слова через свій стан, і на останньому кроці її вихідний шар (зазвичай з функцією Softmax) розраховує ймовірності для кожного слова зі словника. Слово з найвищою ймовірністю (наприклад, "університет") обирається як наступне. На цьому принципі працюють підказки на клавіатурі смартфона.

Висновки

У цій лабораторній роботі було проведено повний цикл розробки, навчання та тестування рекурентної нейронної мережі архітектури LSTM для задачі бінарної класифікації (аналізу настроїв текстів) на базі набору даних Yelp Review Polarity. Під час виконання роботи було успішно реалізовано попередню обробку текстових даних. Ключовим етапом стало проведення порівняльного експерименту з використанням трьох різних методів токенизації: Word-level, Character-level та Subword-level.

На основі отриманих графіків навчання, метрик (Accuracy, Precision, Recall, F1-Score) та матриць помилок встановлено:

1. **Word-level та Subword-level** методи продемонстрували високу точність класифікації (понад 92%) та стабільне навчання. Мережа успішно навчилася розрізняти емоційне забарвлення.
2. **Character-level** метод показав найгірший результат (точність приблизно 55%). Як видно з результатів фінального тестування, ця модель почала ігнорувати вхідні дані, видаючи для будь-якого тексту однакову ймовірність. Це наочно

підтверджує теоретичний недолік RNN/LSTM: вони втрачають здатність утримувати контекст на надзвичайно довгих послідовностях.

Фінальне тестування на власних текстах, які не входили до навчальної вибірки, довело, що навчені моделі мають високу здатність до узагальнення. Отже, робота підтвердила, що в сучасних задачах NLP якість підготовки даних та вибір правильного токенизатора впливають на кінцевий результат не менше, ніж глибина самої архітектури нейронної мережі, а сучасні Subword-методи є найбільш оптимальним рішенням.